

Empirical IO I: Problem Set 0

Chris Conlon

This problem set is designed to make sure that your **numpy** skills are up to speed. Python/**numpy** is the reference implementation below; any language is acceptable, but then it is on you to map each task to the corresponding routines. For each question, the expectation is that you complete the task, provide the appropriate code, and fully read the documentation, and work the **numpy** provided examples on your own. The tasks should be fairly easy. The goal is to accomplish the tasks in the most straightforward manner with the least amount of code.

If you are new to **numpy** I suggest the following tutorials:

- <https://numpy.org/doc/stable/user/quickstart.html>
- <https://numpy.org/doc/stable/user/numpy-for-matlab-users.html>

Part 0: Logit Inclusive Value

The logit inclusive value or $IV = \log \sum_{i=0}^N \exp[x_i]$.

1. Show that this function is everywhere convex.
2. A common problem in practice is that if one of the $x_i > 600$ that we have an “overflow” error on a computer. In this case $\exp[600] \approx 10^{260}$ which is too large to store with any real precision, especially if another x has a different scale (say $x_2 = 10$). A common “trick” is to subtract off $m_i = \max_i x_i$ from all x_i . Show how to implement the trick and get the correct value of IV . If you get stuck take a look at Wikipedia.
3. Compare your function to `scipy.special.logsumexp`. Does it appear to suffer from underflow/overflow? Does it use the max trick?

Part 1: Markov Chains

Consider the following Markov TPM:

Let $P = \{p_{i,j}\}$ be an $n \times n$ transition matrix of a Markov process where $\{p_{i,j}\}$ is interpreted as the probability that the system, when it is in state i , will move to state j . If we denote by $\mu_t = [\mu_{t,1}, \mu_{t,2}, \dots, \mu_{t,n}]$ the probability mass function of the system over the n states then $\mu_{t,j}$ evolves according to

$$\mu_{t+1,j} = \sum_{i=1}^n p_{i,j} \mu_{t,i}$$

Then we can write the state to state transition matrix as :

$$\begin{aligned} \mu_{t+1} &= \mu_t P \\ P &= \begin{bmatrix} 0.2 & 0.4 & 0.4 \\ 0.1 & 0.3 & 0.6 \\ 0.5 & 0.1 & 0.4 \end{bmatrix} \end{aligned}$$

We're interested in the ergodic distribution $\mu P = \mu$. This is similar to the transition matrix infinitely many periods into the future P^∞ . Write a function that computes the ergodic distribution of the matrix P by examining the properly rescaled eigenvectors and compare your result to P^{100} . Here I recommend `numpy.linalg.matrix_power` and `numpy.linalg.eig`. (A common mistake is element-wise exponentiation of the matrix).

Part 2: Numerical Integration

Note: The `scipy` quadrature routines may return a set of nodes/weights that correspond to integrating e^{-x^2} over $(-\infty, \infty)$, while the nodes/weights available at <https://www.sparse-grids.de> may not. It is always important to make sure you understand what your nodes/weights correspond to. One way to do this is to integrate some simple functions $f(x) = 1, f(x) = x, f(x) = x^2$ where you know the analytic result and see what the quadrature routine gives as the answer.

In this part we will look at calculating the logit choice probability $\sigma(X, \theta)$ by numerical integration:

$$\sigma(X, \theta) = \int_{-\infty}^{\infty} \frac{\exp(\beta_i X)}{1 + \exp(\beta_i X)} f(\beta_i | \theta) d\beta_i.$$
Assume $f \sim N(0.5, 2)$ and that $X = 0.5$.

1. Create the function in an Python called `binomiallogit`. (It should take β the item you integrate over as its argument, it should take the PDF `scipy.stats.norm.pdf` as an optional argument).
2. Integrate the function using Python's `scipy.integrate.quad` command and setting the tolerance to 1×10^{-14} . Treat this as the "true" value.
3. Integrate the function by taking 20 and 400 Monte Carlo draws from f and computing the sample mean.
4. Integrate the function using Gauss-Hermite quadrature for $k = 4, 12$ (Try some odd ones too). Obtain the quadrature nodes and weights from `numpy.polynomial.hermite.hermgauss`. Gauss-Hermite quadrature assumes a weighting function of $\exp[-x^2]$, you will need a change of variables to integrate over a normal density (see the Numerical Integration notes in **Extra Notes/**). You also need to pay attention to the constant of integration.
5. Compare results to the Monte Carlo results. *Make sure your quadrature weights sum to 1!*
6. Repeat the exercise in two dimensions where $\mu = (0.5, 1), \sigma = (2, 1)$, and $X = (0.5, 1)$.
7. Put everything into two tables (one for the 1-D integral, one for the 2-D integral). Showing the error from the "true" value and the number of points used in the evaluation.
8. Now Construct a new function `binomiallogitmixture` that takes a vector for X and returns a vector of binomial probabilities (appropriately integrated over $f(\beta_i | \theta)$ for the 1-D mixture). It should be obvious that Gauss-Hermite is the most efficient way to do this. *Do NOT use loops*